

LAPORAN PRAKTIKUM

Struktur Data

“BinaryTree dan graph”

DISUSUN OLEH:

ABDUL JABBAR

2511533021

DOSEN PENGAMPU:

Dr. WAHYUDI, S.T, M.T

ASISTEN PRAKTIKUM:

SHERLY SUKMADIRA



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji dan syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya laporan praktikum mengenai Binary Tree dan Graph ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu tugas praktikum Struktur Data yang bertujuan untuk memahami konsep, cara kerja, serta implementasi struktur data Binary Tree dan Graph dalam pemrograman.

Binary Tree merupakan salah satu struktur data non-linear yang banyak digunakan untuk menyimpan dan mengelola data secara hierarkis sehingga proses pencarian, penyisipan, dan penghapusan data dapat dilakukan dengan lebih efisien. Sementara itu, Graph merupakan struktur data yang digunakan untuk merepresentasikan hubungan antar objek melalui simpul (vertex) dan sisi (edge), yang banyak diterapkan dalam berbagai bidang seperti jaringan komputer, navigasi, dan media sosial.

Melalui praktikum ini, penulis dapat mempelajari konsep dasar, algoritma, serta implementasi Binary Tree dan Graph dalam pemecahan berbagai permasalahan komputasi. Selain itu, praktikum ini juga memberikan pengalaman dalam menerapkan teori yang telah dipelajari ke dalam program yang dapat dijalankan dan dianalisis hasilnya.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan baik dari segi penyajian maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun demi perbaikan laporan di masa yang akan datang.

Padang, Mei 2026

Abdul Jabbar

DAFTAR ISI

KATA PENGANTAR.....	ii
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
BAB II PEMBAHASAN	3
2.1 Pengertian	3
2.2 Praktikum pekan 9.....	3
2.3.1 Node	4
2.3.2 BTree.....	6
2.3.3 BTreeDriver.....	8
2.3.4 GraphTraversal	10
2.3.5 Laporan tugas_2511533021.....	12
DAFTAR PUSTAKA	19

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan ilmu komputer, struktur data memiliki peran yang sangat penting dalam mengelola serta menyimpan data secara efisien. Pemilihan struktur data yang tepat dapat meningkatkan kinerja suatu program, terutama dalam proses pencarian, penyisipan, penghapusan, dan pengolahan data. Oleh karena itu, pemahaman mengenai berbagai jenis struktur data menjadi hal yang wajib bagi seorang programmer.

Salah satu struktur data non-linear yang sering digunakan adalah Binary Tree. Binary Tree merupakan struktur data berbentuk pohon yang setiap simpulnya memiliki maksimal dua anak, yaitu anak kiri dan anak kanan. Struktur ini banyak digunakan dalam berbagai aplikasi komputer karena mampu mempercepat proses pencarian dan pengelolaan data, terutama pada Binary Search Tree (BST).

Selain Binary Tree, terdapat juga struktur data Graph yang digunakan untuk merepresentasikan hubungan antar objek. Graph terdiri dari simpul (vertex) dan sisi (edge) yang menghubungkan simpul-simpul tersebut. Struktur data ini banyak diterapkan dalam kehidupan sehari-hari, seperti pada jaringan komputer, sistem navigasi, media sosial, dan pemetaan jalur transportasi.

Melalui praktikum ini, mahasiswa diharapkan dapat memahami konsep dasar Binary Tree dan Graph, mengetahui cara kerja kedua struktur data tersebut, serta mampu mengimplementasikannya dalam program komputer untuk menyelesaikan berbagai permasalahan yang berkaitan dengan pengelolaan data dan hubungan antar objek.

1.2 Tujuan

1. Memahami konsep dasar struktur data Binary Tree dan Graph.

2. Mengetahui cara kerja Binary Tree dalam menyimpan dan mengelola data secara hierarkis.
3. Memahami konsep simpul (vertex) dan sisi (edge) pada Graph.
4. Mempelajari implementasi Binary Tree dan Graph menggunakan bahasa pemrograman.
5. Meningkatkan kemampuan dalam menganalisis dan menyelesaikan masalah menggunakan struktur data non-linear.

1.3 Manfaat

1. Menambah pemahaman mengenai struktur data non-linear yang banyak digunakan dalam pengembangan perangkat lunak.
2. Melatih kemampuan dalam mengimplementasikan Binary Tree dan Graph ke dalam program komputer.
3. Membantu memahami cara pengelolaan data yang lebih efisien melalui struktur pohon dan graf.
4. Meningkatkan kemampuan berpikir logis dan analitis dalam menyelesaikan permasalahan komputasi.
5. Menjadi bekal dalam mempelajari algoritma dan struktur data yang lebih kompleks pada mata kuliah lanjutan.
6. Memahami penerapan Binary Tree dan Graph dalam berbagai bidang seperti jaringan komputer, sistem pencarian data, navigasi, dan pemetaan hubungan antar objek.

BAB II

PEMBAHASAN

2.1 Pengertian

Dalam dunia pemrograman dan ilmu komputer, memahami struktur data adalah langkah penting untuk membangun sistem yang efisien dan terorganisir. Setelah mempelajari array, linked list, stack, queue, hingga sorting dan searching algorithm, tahap berikutnya adalah mengenal struktur data non-linear seperti Tree dan Graph.

Berbeda dengan struktur data linear yang menyusun elemen secara berurutan, tree dan graph memungkinkan hubungan data yang lebih kompleks dan fleksibel. Keduanya banyak digunakan dalam sistem modern seperti database, kecerdasan buatan, jaringan komputer, hingga media sosial. Struktur data non-linear adalah struktur data yang tidak menyusun elemen secara berurutan seperti array atau linked list. Dalam struktur ini, satu elemen bisa terhubung ke lebih dari satu elemen lainnya.

Tree adalah struktur data berbentuk hierarki yang terdiri dari kumpulan node (simpul) yang saling terhubung. Struktur ini menyerupai pohon terbalik, dengan satu akar (root) di bagian atas dan cabang-cabang di bawahnya.

2.2 Praktikum pekan 9

Pada praktikum Pekan 9, mahasiswa Informatika Universitas Andalas mempelajari konsep struktur data non-linear yaitu Binary Tree dan Graph dengan membuat, mengimplementasikan, serta menganalisis kode program menggunakan aplikasi Eclipse IDE. Praktikum ini bertujuan untuk memahami cara kerja Binary Tree dalam menyimpan data secara hierarkis dan Graph dalam merepresentasikan hubungan antar objek melalui simpul (vertex) dan sisi (edge). Melalui kegiatan praktikum ini, mahasiswa diharapkan mampu memahami konsep dasar, operasi-operasi yang dapat dilakukan pada Binary Tree dan Graph, serta penerapannya dalam penyelesaian berbagai permasalahan komputasi.

2.3.1 Node

```
1 package Pekan9_2511533021;
2
3 public class Node_2511533021 {
4     int data_3021;
5     Node_2511533021 left_3021;
6     Node_2511533021 right_3021;
7
8     public Node_2511533021(int data_3021) {
9         this.data_3021 = data_3021;
10        left_3021 = null;
11        right_3021 = null;
12    }
13
14    public void setLeft_3021(Node_2511533021 node_3021) {
15        if (left_3021 == null)
16            left_3021 = node_3021;
17    }
18
19    public void setRight_3021(Node_2511533021 node_3021) {
20        if (right_3021 == null)
21            right_3021 = node_3021;
22    }
23
24    public Node_2511533021 getLeft_3021() {
25        return left_3021;
26    }
27
28    public Node_2511533021 getRight_3021() {
29        return right_3021;
30    }
31
32    public int getData_3021() {
33        return data_3021;
34    }
35
36    public void setData_3021() {
37        this.data_3021 = data_3021;
38    }
39
40    void printPreorder_3021(Node_2511533021 node_3021) {
41        if (node_3021 == null)
42            return;
43
44        System.out.println(node_3021.data_3021 + " ");
45        printPreorder_3021(node_3021.left_3021);
46        printPreorder_3021(node_3021.right_3021);
47    }
48
49    void printPostorder_3021(Node_2511533021 node_3021) {
50        if (node_3021 == null)
51            return;
52
53        printPostorder_3021(node_3021.left_3021);
54        printPostorder_3021(node_3021.right_3021);
55        System.out.print(node_3021.data_3021 + " ");
56    }
57
58    void printInorder_3021(Node_2511533021 node_3021) {
59        if (node_3021 == null)
60            return;
61
62        printInorder_3021(node_3021.left_3021);
63        System.out.print(node_3021.data_3021 + " ");
64        printInorder_3021(node_3021.right_3021);
65    }
66
67    public String print_3021() {
68        return this.print_3021("", true, "");
69    }
70
71    public String print_3021(String prefix_3021, boolean isTail_3021, String sb_3021) {
72        if (right_3021 != null) {
73            right_3021.print_3021(
74                prefix_3021 + (isTail_3021 ? " | " : " : "),
75                false,
76                sb_3021
77            );
78        }
79        System.out.println(
80            prefix_3021 + (isTail_3021 ? "\\--" : "/--") + data_3021
81        );
82
83        if (left_3021 != null) {
84            left_3021.print_3021(
85                prefix_3021 + (isTail_3021 ? " : " : " | "),
86                true,
87                sb_3021
88            );
89        }
90        return sb_3021;
91    }
92 }
93 }
```

Program ini merupakan implementasi struktur data Binary Tree menggunakan bahasa Java. Kelas Node_2511533021 digunakan untuk merepresentasikan sebuah node yang memiliki data, anak kiri (left), dan anak kanan (right).

Fungsi Utama Program:

- Menyimpan data pada setiap node tree.
- Menghubungkan node dengan anak kiri dan kanan.
- Menampilkan isi tree menggunakan traversal Preorder, Inorder, dan Postorder.
- Menampilkan bentuk struktur tree secara visual.

Atribut:

- data_3021 : menyimpan nilai pada node.
- left_3021 : menunjuk ke anak kiri.
- right_3021 : menunjuk ke anak kanan.

Method:

- setLeft_3021() : menambahkan anak kiri.
- setRight_3021() : menambahkan anak kanan.
- getLeft_3021() : mengambil anak kiri.
- getRight_3021() : mengambil anak kanan.
- getData_3021() : mengambil data node.
- printPreorder_3021() : menampilkan data dengan urutan Root → Left → Right.
- printInorder_3021() : menampilkan data dengan urutan Left → Root → Right.
- printPostorder_3021() : menampilkan data dengan urutan Left → Right → Root.
- print_3021() : menampilkan bentuk visual Binary Tree.

2.3.2 BTree

```
1 package Pkan9_2511533021;
2
3 public class BTree_2511533021 {
4     private Node_2511533021 root_3021;
5     private Node_2511533021 currentNode_3021;
6
7     public BTree_2511533021() {
8         root_3021 = null;
9     }
10
11     public boolean search_3021(int data_3021) {
12         return search_3021(root_3021, data_3021);
13     }
14
15     private boolean search_3021(Node_2511533021 node_3021, int data_3021) {
16         if (node_3021.getData_3021() == data_3021)
17             return true;
18
19         if (node_3021.getLeft_3021() != null)
20             if (search_3021(node_3021.getLeft_3021(), data_3021))
21                 return true;
22
23         if (node_3021.getRight_3021() != null)
24             if (search_3021(node_3021.getRight_3021(), data_3021))
25                 return true;
26
27         return false;
28     }
29
30     public void printInOrder_3021() {
31         root_3021.printInOrder_3021(root_3021);
32     }
33
34     public void printPreOrder_3021() {
35         root_3021.printPreOrder_3021(root_3021);
36     }
37
38     public void printPostOrder_3021() {
39         root_3021.printPostOrder_3021(root_3021);
40     }
41
42     public Node_2511533021 getRoot_3021() {
43         return root_3021;
44     }
45
46     public boolean isEmpty_3021() {
47         return root_3021 == null;
48     }
49
50     public int countNodes_3021() {
51         return countNodes_3021(root_3021);
52     }
53
54     private int countNodes_3021(Node_2511533021 node_3021) {
55         int count_3021 = 1;
56
57         if (node_3021 == null) {
58             return 0;
59         } else {
60             count_3021 += countNodes_3021(node_3021.getLeft_3021());
61             count_3021 += countNodes_3021(node_3021.getRight_3021());
62             return count_3021;
63         }
64     }
65
66     public void print_3021() {
67         root_3021.print_3021();
68     }
69
70     public Node_2511533021 getCurrent_3021() {
71         return currentNode_3021;
72     }
73 }
```

Program ini merupakan implementasi Binary Tree yang berfungsi untuk mengelola kumpulan node pada struktur pohon biner. Kelas BTree_2511533021 digunakan sebagai pengendali utama yang mengatur root, pencarian data, perhitungan jumlah node, dan proses traversal.

Fungsi Utama Program:

- Menyimpan root dari Binary Tree.
- Melakukan pencarian data pada seluruh node tree.
- Menghitung jumlah node yang terdapat dalam tree.
- Menampilkan isi tree menggunakan traversal Inorder, Preorder, dan Postorder.

- Menampilkan struktur Binary Tree secara visual.
- Mengecek apakah tree kosong atau tidak.

Atribut:

- root_3021 : node akar (root) dari Binary Tree.
- currentNode_3021 : node yang sedang aktif atau diproses.

Method:

- search_3021() : mencari data tertentu dalam tree.
- printInorder_3021() : menampilkan traversal Inorder (Left → Root → Right).
- printPreOrder_3021() : menampilkan traversal Preorder.
- printPostOrder_3021() : menampilkan traversal Postorder.
- getRoot_3021() : mengambil root tree.
- setRoot_3021() : mengatur root tree.
- isEmpty_3021() : memeriksa apakah tree kosong.
- countNodes_3021() : menghitung jumlah seluruh node dalam tree.
- print_3021() : menampilkan struktur tree secara visual.
- getCurrent_3021() : mengambil node yang sedang aktif.
- setCurrent_3021() : mengatur node yang sedang aktif.

2.3.3 BTreeDriver

```
1 package Pekan9_2511533021;
2
3 public class BTreeDriver_2511533021 {
4     public static void main(String[] args) {
5         // membuat pohon
6         BTree_2511533021 tree_3021 = new BTree_2511533021();
7
8         System.out.println("Jumlah Simpul awal pohon: ");
9         System.out.println(tree_3021.countNodes_3021());
10
11         // menambahkan simpul data 1
12         Node_2511533021 root_3021 = new Node_2511533021(1);
13
14         // menjadikan simpul 1 sebagai root
15         tree_3021.setRoot_3021(root_3021);
16
17         System.out.println("Jumlah simpul jika hanya ada root");
18         System.out.println(tree_3021.countNodes_3021());
19
20         Node_2511533021 node2_3021 = new Node_2511533021(2);
21         Node_2511533021 node3_3021 = new Node_2511533021(3);
22         Node_2511533021 node4_3021 = new Node_2511533021(4);
23         Node_2511533021 node5_3021 = new Node_2511533021(5);
24         Node_2511533021 node6_3021 = new Node_2511533021(6);
25         Node_2511533021 node7_3021 = new Node_2511533021(7);
26         Node_2511533021 node8_3021 = new Node_2511533021(8);
27         Node_2511533021 node9_3021 = new Node_2511533021(9);
28
29         root_3021.setLeft_3021(node2_3021);
30         node2_3021.setLeft_3021(node4_3021);
31         node2_3021.setRight_3021(node5_3021);
32         node4_3021.setRight_3021(node6_3021);
33
34         root_3021.setRight_3021(node3_3021);
35         node3_3021.setLeft_3021(node6_3021);
36         node3_3021.setRight_3021(node7_3021);
37         node6_3021.setLeft_3021(node9_3021);
38
39         // set root
40         tree_3021.setCurrent_3021(tree_3021.getRoot_3021());
41
42         System.out.println("menampilkan simpul terakhir:");
43         System.out.println(tree_3021.getCurrent_3021().getData_3021());
44
45         System.out.println("Jumlah simpul; setelah simpul 7 ditambahkan");
46         System.out.println(tree_3021.countNodes_3021());
47
48         System.out.println("InOrder: ");
49         tree_3021.printInOrder_3021();
50
51         System.out.println("\nPreorder: ");
52         tree_3021.printPreOrder_3021();
53
54         System.out.println("\nPostorder: ");
55         tree_3021.printPostOrder_3021();
56
57         System.out.println("\nMenampilkan simpul dalam bentuk pohon");
58         tree_3021.print_3021();
59     }
60 }
```

Program BTreeDriver_2511533021 berfungsi sebagai kelas utama (driver) untuk menguji implementasi Binary Tree. Pada program ini dibuat beberapa node yang kemudian dihubungkan menjadi sebuah pohon biner dan dilakukan berbagai operasi seperti menghitung jumlah node serta menampilkan traversal tree.

Fungsi Utama Program:

- Membuat objek Binary Tree.
- Menambahkan node ke dalam tree.
- Menentukan root tree.
- Menghubungkan node sebagai anak kiri dan anak kanan.
- Menghitung jumlah node pada tree.
- Menampilkan traversal Inorder, Preorder, dan Postorder.
- Menampilkan bentuk visual Binary Tree.

Alur Program:

1. Membuat objek Binary Tree.
2. Membuat node dengan nilai 1 sampai 9.
3. Menjadikan node 1 sebagai root.
4. Menghubungkan node-node sehingga membentuk Binary Tree.
5. Menghitung jumlah node yang ada dalam tree.
6. Menampilkan traversal Inorder, Preorder, dan Postorder.
7. Menampilkan struktur tree dalam bentuk pohon.

2.3.4 GraphTraversal

```
1 package Ppkan9_2511533021;
2
3 import java.util.*;
4
5 public class GraphTraversal_2511533021 {
6     private Map<String, List<String>> graph_3021 = new HashMap<>();
7
8     // Menambahkan edge (graf tak berarah)
9     public void addEdge(String node1, String node2) {
10         graph_3021.putIfAbsent(node1, new ArrayList<>());
11         graph_3021.putIfAbsent(node2, new ArrayList<>());
12         graph_3021.get(node1).add(node2);
13         graph_3021.get(node2).add(node1);
14     }
15
16     // Menampilkan graf awal
17     public void printGraph_3021() {
18         System.out.println("Graf Awal (Adjacency List):");
19         for (String node_3021 : graph_3021.keySet()) {
20             System.out.print(node_3021 + " -> ");
21             List<String> neighbors_3021 = graph_3021.get(node_3021);
22             System.out.println(String.join(", ", neighbors_3021));
23         }
24         System.out.println();
25     }
26
27     // DFS Rekursif
28     public void dfs_3021(String start_3021) {
29         Set<String> visited_3021 = new HashSet<>();
30         System.out.println("Penelusuran DFS:");
31         dfsHelper_3021(start_3021, visited_3021);
32         System.out.println();
33     }
34
35     private void dfsHelper_3021(String current_3021, Set<String> visited_3021) {
36         if (visited_3021.contains(current_3021))
37             return;
38
39         visited_3021.add(current_3021);
40         System.out.print(current_3021 + " ");
41
42         for (String neighbor_3021 : graph_3021.getOrDefault(current_3021, new ArrayList<>())) {
43             dfsHelper_3021(neighbor_3021, visited_3021);
44         }
45     }
46
47     // BFS Iteratif
48     public void bfs_3021(String start_3021) {
49         Set<String> visited_3021 = new HashSet<>();
50         Queue<String> queue_3021 = new LinkedList<>();
51
52         queue_3021.add(start_3021);
53         visited_3021.add(start_3021);
54
55         System.out.println("Penelusuran BFS:");
56
57         while (!queue_3021.isEmpty()) {
58             String current_3021 = queue_3021.poll();
59             System.out.print(current_3021 + " ");
60
61             for (String neighbor_3021 : graph_3021.getOrDefault(current_3021, new ArrayList<>())) {
62                 if (!visited_3021.contains(neighbor_3021)) {
63                     queue_3021.add(neighbor_3021);
64                     visited_3021.add(neighbor_3021);
65                 }
66             }
67         }
68         System.out.println();
69     }
70
71     // Main
72     public static void main(String[] args) {
73         GraphTraversal_2511533021 graph_3021 = new GraphTraversal_2511533021();
74
75         // Contoh graf: A-B, A-C, B-D, B-E
76         graph_3021.addEdge("A", "B");
77         graph_3021.addEdge("A", "C");
78         graph_3021.addEdge("B", "D");
79         graph_3021.addEdge("B", "E");
80
81         // Cetak graf awal
82         System.out.println("Graf Awal adalah:");
83         graph_3021.printGraph_3021();
84
85         // Lakukan penelusuran
86         graph_3021.dfs_3021("A");
87         graph_3021.bfs_3021("A");
88     }
89 }
```

Program GraphTraversal_2511533021 merupakan implementasi struktur data Graph menggunakan Java. Program ini digunakan untuk membuat graf tak berarah (Undirected Graph) serta melakukan penelusuran menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS).

Fungsi Utama Program:

- Membuat graf menggunakan Adjacency List.
- Menambahkan hubungan (edge) antar simpul.
- Menampilkan struktur graf.
- Melakukan penelusuran DFS secara rekursif.
- Melakukan penelusuran BFS secara iteratif menggunakan Queue.

Struktur Data yang Digunakan:

- HashMap untuk menyimpan simpul dan daftar tetangganya.
- ArrayList untuk menyimpan node yang terhubung.
- HashSet untuk menandai simpul yang telah dikunjungi.
- Queue (LinkedList) untuk proses BFS.

Method:

- addEdge() : menambahkan hubungan antara dua simpul.
- printGraph_3021() : menampilkan graf dalam bentuk adjacency list.
- dfs_3021() : menjalankan penelusuran DFS.
- dfsHelper_3021() : method rekursif untuk DFS.
- bfs_3021() : menjalankan penelusuran BFS.

Alur Program:

1. Membuat graf dengan simpul A, B, C, D, dan E.
2. Menambahkan edge A-B, A-C, B-D, dan B-E.
3. Menampilkan graf dalam bentuk adjacency list.
4. Melakukan penelusuran DFS dimulai dari simpul A.
5. Melakukan penelusuran BFS dimulai dari simpul A.

2.3.5 Laporan tugas_2511533021

```
1 package Pekan9_2511533021;
2
3 import java.awt.*;
4 import java.awt.event.*;
5 import java.util.*;
6 import java.util.List;
7 import javax.swing.*;
8 import javax.swing.border.*;
9
10 public class PetaTempatWisata_2511533021 extends JFrame {
11
12     private static final long serialVersionUID = 1L;
13
14     static class GraphTraversal_3021 {
15
16         private Map<String, List<String>> graph_3021 = new LinkedHashMap<>();
17
18         public void addEdge_3021(String node1_3021, String node2_3021) {
19             graph_3021.putIfAbsent(node1_3021, new ArrayList<>());
20             graph_3021.putIfAbsent(node2_3021, new ArrayList<>());
21             graph_3021.get(node1_3021).add(node2_3021);
22             graph_3021.get(node2_3021).add(node1_3021);
23         }
24
25         public List<String[]> getAllEdges_3021() {
26             List<String[]> edges_3021 = new ArrayList<>();
27             Set<String> added_3021 = new HashSet<>();
28             for (String node_3021 : graph_3021.keySet()) {
29                 for (String neighbor_3021 : graph_3021.get(node_3021)) {
30                     String key1_3021 = node_3021 + "-" + neighbor_3021;
31                     String key2_3021 = neighbor_3021 + "-" + node_3021;
32                     if (!added_3021.contains(key1_3021) && !added_3021.contains(key2_3021)) {
33                         edges_3021.add(new String[]{node_3021, neighbor_3021});
34                         added_3021.add(key1_3021);
35                         added_3021.add(key2_3021);
36                     }
37                 }
38             }
39             return edges_3021;
40         }
41
42         // BFS iteratif
43         public TraversalResult_3021 bfs_3021(String start_3021, String goal_3021) {
44             List<String> visited_3021 = new ArrayList<>();
45             Map<String, String> parent_3021 = new LinkedHashMap<>();
46             Queue<String> queue_3021 = new LinkedList<>();
47             queue_3021.add(start_3021);
48             visited_3021.add(start_3021);
49             parent_3021.put(start_3021, null);
50             while (!queue_3021.isEmpty()) {
51                 String current_3021 = queue_3021.poll();
52                 if (current_3021.equals(goal_3021)) break;
53                 for (String neighbor_3021 : graph_3021.getOrDefault(current_3021, new ArrayList<>())) {
54                     if (!visited_3021.contains(neighbor_3021)) {
55                         queue_3021.add(neighbor_3021);
56                         visited_3021.add(neighbor_3021);
57                         parent_3021.put(neighbor_3021, current_3021);
58                     }
59                 }
60             }
61             return new TraversalResult_3021(visited_3021, buildPath_3021(parent_3021, start_3021, goal_3021));
62         }
63
64         // DFS rekursif
65         public TraversalResult_3021 dfs_3021(String start_3021, String goal_3021) {
66             List<String> visited_3021 = new ArrayList<>();
67             Map<String, String> parent_3021 = new LinkedHashMap<>();
68             parent_3021.put(start_3021, null);
69             dfsHelper_3021(start_3021, goal_3021, visited_3021, parent_3021);
70             return new TraversalResult_3021(visited_3021, buildPath_3021(parent_3021, start_3021, goal_3021));
71         }
72
73         private boolean dfsHelper_3021(String current_3021, String goal_3021,
74                                     List<String> visited_3021, Map<String, String> parent_3021) {
75             if (visited_3021.contains(current_3021)) return false;
76             visited_3021.add(current_3021);
77             if (current_3021.equals(goal_3021)) return true;
78             for (String neighbor_3021 : graph_3021.getOrDefault(current_3021, new ArrayList<>())) {
79                 if (!visited_3021.contains(neighbor_3021)) {
80                     parent_3021.put(neighbor_3021, current_3021);
81                     if (dfsHelper_3021(neighbor_3021, goal_3021, visited_3021, parent_3021)) return true;
82                 }
83             }
84             return false;
85         }
86
87         private List<String> buildPath_3021(Map<String, String> parent_3021,
88                                           String start_3021, String goal_3021) {
89             List<String> path_3021 = new ArrayList<>();
90             if (!parent_3021.containsKey(goal_3021)) return path_3021;
91             String cur_3021 = goal_3021;
92             while (cur_3021 != null) {
93                 path_3021.add(0, cur_3021);
94                 cur_3021 = parent_3021.get(cur_3021);
95             }
96             return path_3021;
97         }
98
99         static class TraversalResult_3021 {
100             public final List<String> visited_3021;
101             public final List<String> path_3021;
```

```

101     TraversalResult_3021(List<String> v, List<String> p) {
102         visited_3021 = v;
103         path_3021 = p;
104     }
105 }
106
107
108
109 class GraphPanel_3021 extends JPanel {
110
111     private static final long serialVersionUID = 2L;
112     private static final int R_3021 = 30;
113
114     @Override
115     protected void paintComponent(Graphics g_3021) {
116         super.paintComponent(g_3021);
117         Graphics2D g2_3021 = (Graphics2D) g_3021;
118         g2_3021.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
119                                 RenderingHints.VALUE_ANTIALIAS_ON);
120
121         drawEdges_3021(g2_3021);
122         drawNodes_3021(g2_3021);
123         drawLegend_3021(g2_3021);
124     }
125
126     private void drawEdges_3021(Graphics2D g2_3021) {
127         for (String[] edge_3021 : traversal_3021.getAllEdges_3021()) {
128             int[] p1_3021 = nodePos_3021.get(edge_3021[0]);
129             int[] p2_3021 = nodePos_3021.get(edge_3021[1]);
130             if (p1_3021 == null || p2_3021 == null) continue;
131             if (isEdgeOnPath_3021(edge_3021[0], edge_3021[1])) {
132                 g2_3021.setColor(Color.ORANGE);
133                 g2_3021.setStroke(new BasicStroke(3f));
134             } else {
135                 g2_3021.setColor(Color.DARK_GRAY);
136                 g2_3021.setStroke(new BasicStroke(1.5f));
137             }
138             g2_3021.drawLine(p1_3021[0], p1_3021[1], p2_3021[0], p2_3021[1]);
139             g2_3021.setStroke(new BasicStroke(1f));
140         }
141
142     private boolean isEdgeOnPath_3021(String a_3021, String b_3021) {
143         if (pathNodes_3021.size() < 2) return false;
144         for (int i_3021 = 0; i_3021 < pathNodes_3021.size() - 1; i_3021++) {
145             String u_3021 = pathNodes_3021.get(i_3021);
146             String v_3021 = pathNodes_3021.get(i_3021 + 1);
147             if ((u_3021.equals(a_3021) && v_3021.equals(b_3021)) ||
148                 (u_3021.equals(b_3021) && v_3021.equals(a_3021))) return true;
149         }
150         return false;
151     }
152
153     private void drawNodes_3021(Graphics2D g2_3021) {
154         for (String node_3021 : nodePos_3021.keySet()) {
155             int[] pos_3021 = nodePos_3021.get(node_3021);
156             int x_3021 = pos_3021[0];
157             int y_3021 = pos_3021[1];
158
159             Color fill_3021;
160             if (pathNodes_3021.isEmpty() && node_3021.equals(pathNodes_3021.get(0))) {
161                 fill_3021 = Color.GREEN;
162             } else if (pathNodes_3021.isEmpty() &&
163                      node_3021.equals(pathNodes_3021.get(pathNodes_3021.size() - 1))) {
164                 fill_3021 = Color.RED;
165             } else if (pathNodes_3021.contains(node_3021)) {
166                 fill_3021 = Color.YELLOW;
167             } else if (visitedNodes_3021.contains(node_3021)) {
168                 fill_3021 = Color.CYAN;
169             } else {
170                 fill_3021 = Color.LIGHT_GRAY;
171             }
172
173             g2_3021.setColor(fill_3021);
174             g2_3021.fillOval(x_3021 - R_3021, y_3021 - R_3021, R_3021 * 2, R_3021 * 2);
175             g2_3021.setColor(Color.BLACK);
176             g2_3021.setStroke(new BasicStroke(1.5f));
177             g2_3021.drawOval(x_3021 - R_3021, y_3021 - R_3021, R_3021 * 2, R_3021 * 2);
178             g2_3021.setStroke(new BasicStroke(1f));
179
180             g2_3021.setFont(new Font("SansSerif", Font.BOLD, 10));
181             g2_3021.setColor(Color.BLACK);
182             FontMetrics fm_3021 = g2_3021.getFontMetrics();
183             String[] parts_3021 = node_3021.split(" ", 2);
184             if (parts_3021.length == 2) {
185                 g2_3021.drawString(parts_3021[0],
186                                   x_3021 - fm_3021.stringWidth(parts_3021[0]) / 2, y_3021 - 3);
187                 g2_3021.drawString(parts_3021[1],
188                                   x_3021 - fm_3021.stringWidth(parts_3021[1]) / 2, y_3021 + 11);
189             } else {
190                 g2_3021.drawString(node_3021,
191                                   x_3021 - fm_3021.stringWidth(node_3021) / 2, y_3021 + 4);
192             }
193         }
194     }
195
196     private void drawLegend_3021(Graphics2D g2_3021) {
197         int lx_3021 = 16;
198         int ly_3021 = getHeight() - 105;
199         g2_3021.setFont(new Font("SansSerif", Font.PLAIN, 11));

```

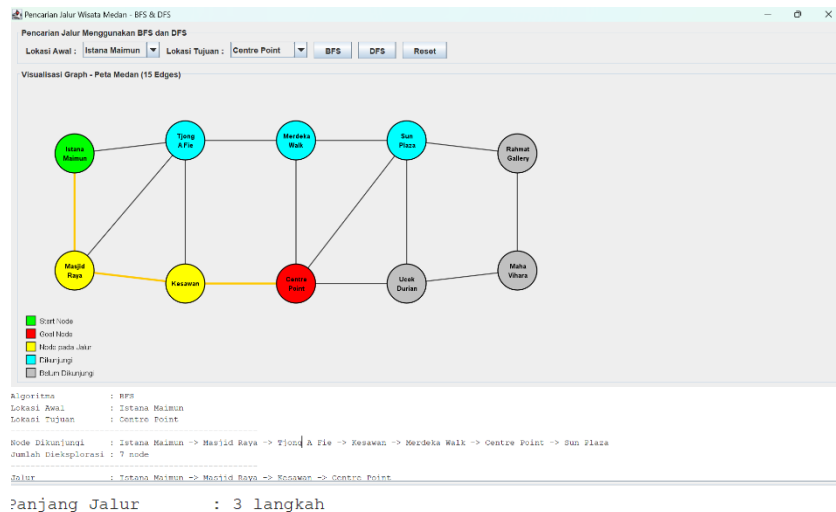


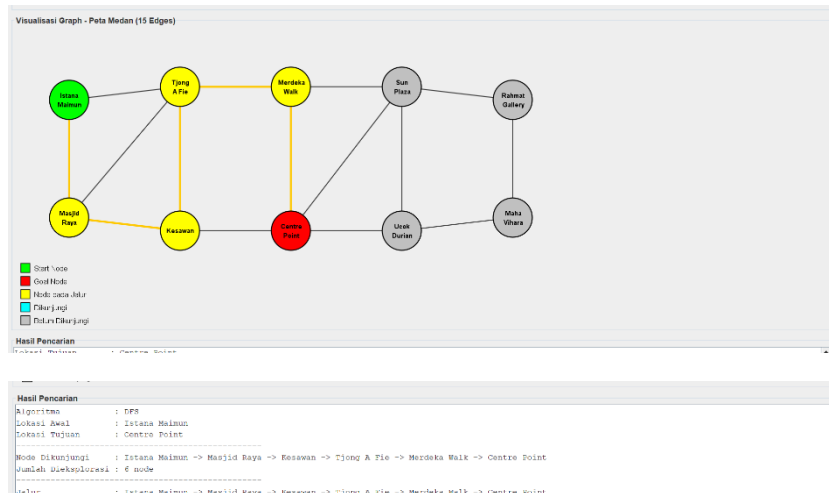
```

199     g2_3021.setFont(new Font("SansSerif", Font.PLAIN, 11));
200     String[][] items_3021 = {
201         {"Start Node", "GREEN"},
202         {"Goal Node", "RED"},
203         {"Node pada Jalur", "YELLOW"},
204         {"Dikunjungi", "CYAN"},
205         {"Belum dikunjungi", "LIGHT_GRAY"}
206     };
207     Color[] colors_3021 = {Color.GREEN, Color.RED, Color.YELLOW, Color.CYAN, Color.LIGHT_GRAY};
208     for (int i_3021 = 0; i_3021 < items_3021.length; i_3021++) {
209         g2_3021.setColor(colors_3021[i_3021]);
210         g2_3021.fillRect(1x_3021, 1y_3021, 14, 14);
211         g2_3021.setColor(Color.BLACK);
212         g2_3021.drawRect(1x_3021, 1y_3021, 14, 14);
213         g2_3021.drawString(items_3021[i_3021][0], 1x_3021 + 20, 1y_3021 + 12);
214         1y_3021 += 20;
215     }
216 }
217 }
218
219
220 private JComboBox<String> cbStart_3021;
221 private JComboBox<String> cbGoal_3021;
222 private JTextArea taResult_3021;
223 private GraphPanel_3021 graphPanel_3021;
224
225 private final GraphTraversal_3021 traversal_3021 = new GraphTraversal_3021();
226 private List<String> visitedNodes_3021 = new ArrayList<>();
227 private List<String> pathNodes_3021 = new ArrayList<>();
228 private final Map<String, int[]> nodePos_3021 = new LinkedHashMap<>();
229
230 private static final String[] NODES_3021 = {
231     "Istana Maimun", "Masjid Raya", "Tjong A Fie", "Kesawan", "Merdeka Walk",
232
367 private void displayResult_3021(String algo_3021, String start_3021, String goal_3021,
368     List<String> visited_3021, List<String> path_3021) {
369     StringBuilder sb_3021 = new StringBuilder();
370     sb_3021.append("Algoritma : ").append(algo_3021).append("\n");
371     sb_3021.append("Lokasi Awal : ").append(start_3021).append("\n");
372     sb_3021.append("Lokasi Tujuan : ").append(goal_3021).append("\n");
373     sb_3021.append("-----\n");
374     sb_3021.append("Node Dikunjungi : ").append(String.join(" -> ", visited_3021)).append("\n");
375     sb_3021.append("Jumlah Dieksplorasi : ").append(visited_3021.size()).append(" node\n");
376     sb_3021.append("-----\n");
377     if (path_3021.isEmpty()) {
378         sb_3021.append("Jalur : Tidak ditemukan!\n");
379     } else {
380         sb_3021.append("Jalur : ").append(String.join(" -> ", path_3021)).append("\n");
381         sb_3021.append("Panjang Jalur : ").append(path_3021.size() - 1).append(" langkah\n");
382     }
383     taResult_3021.setText(sb_3021.toString());
384 }
385
386 private void showSameNodeWarning_3021() {
387     JOptionPane.showMessageDialog(this,
388         "Lokasi awal dan tujuan tidak boleh sama!",
389         "Peringatan", JOptionPane.WARNING_MESSAGE);
390 }
391 }

```

OUTPUT





1. Program dijalankan melalui method `main()`, kemudian membuat objek `PetaTempatWisata_2511533021` dan menampilkan jendela aplikasi.
2. Constructor `PetaTempatWisata_2511533021` dipanggil untuk:
 - Mengatur judul dan ukuran `JFrame`.
 - Memanggil `buildGraph_3021()` untuk membangun graph.
 - Memanggil `setupUI_3021()` untuk membuat tampilan GUI.
3. Method `buildGraph_3021()` membuat graph tempat wisata di Kota Medan:
 - Menentukan posisi setiap node (tempat wisata).
 - Menambahkan 10 node wisata.
 - Menambahkan 15 edge sebagai jalur penghubung antar lokasi.
4. Method `setupUI_3021()` membangun antarmuka pengguna:
 - `ComboBox` untuk memilih lokasi awal dan tujuan.
 - Tombol `BFS`, `DFS`, dan `Reset`.
 - Panel visualisasi graph.
 - `TextArea` untuk menampilkan hasil pencarian.

5. Ketika tombol BFS ditekan:

- Method runBFS_3021() dijalankan.
- Mengambil lokasi awal dan tujuan.
- Memanggil algoritma bfs_3021().
- BFS menggunakan Queue (antrian) untuk menelusuri node secara melebar (level demi level).
- Node yang dikunjungi dan jalur yang ditemukan disimpan.
- Hasil ditampilkan pada TextArea dan graph diperbarui.

6. Ketika tombol DFS ditekan:

- Method runDFS_3021() dijalankan.
- Mengambil lokasi awal dan tujuan.
- Memanggil algoritma dfs_3021().
- DFS menggunakan rekursi untuk menelusuri node sedalam mungkin terlebih dahulu.
- Node yang dikunjungi dan jalur yang ditemukan disimpan.
- Hasil ditampilkan pada TextArea dan graph diperbarui.

7. Method buildPath_3021() digunakan untuk membentuk jalur dari lokasi tujuan kembali ke lokasi awal berdasarkan parent node yang diperoleh selama proses BFS atau DFS.

8. GraphPanel_3021 bertugas menggambar visualisasi graph:

- Menggambar node tempat wisata.
- Menggambar edge penghubung.
- Memberi warna berbeda pada node yang dikunjungi dan jalur hasil pencarian.

9. Keterangan warna pada graph:

- Hijau : Node awal (Start).
- Merah : Node tujuan (Goal).
- Kuning : Node yang termasuk jalur hasil pencarian.
- Cyan : Node yang telah dikunjungi.
- Abu-abu: Node yang belum dikunjungi.

10. Tombol Reset menjalankan method `resetGraph_3021()` untuk:

- Menghapus hasil pencarian.
- Mengosongkan node yang dikunjungi dan jalur.
- Mengembalikan pilihan lokasi ke kondisi awal.
- Menggambar ulang graph seperti semula.

BAB III

PENUTUPAN

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa Binary Tree dan Graph merupakan struktur data non-linear yang digunakan untuk mengelola data dan hubungan antar data secara lebih efisien. Binary Tree menyusun data dalam bentuk hierarki dengan node induk dan node anak, sehingga memudahkan proses pencarian, penyisipan, dan traversal data menggunakan metode Preorder, Inorder, dan Postorder.

Sementara itu, Graph digunakan untuk merepresentasikan hubungan antar simpul (vertex) yang saling terhubung melalui sisi (edge). Struktur data ini sangat berguna dalam berbagai aplikasi seperti jaringan komputer, sistem navigasi, dan pemetaan hubungan antar objek.

Melalui praktikum ini, mahasiswa dapat memahami konsep dasar Binary Tree dan Graph, mengimplementasikannya menggunakan bahasa pemrograman Java pada Eclipse IDE, serta mengetahui bagaimana kedua struktur data tersebut digunakan untuk menyelesaikan berbagai permasalahan dalam bidang informatika.

3.2 Saran

1. Mahasiswa sebaiknya memahami terlebih dahulu konsep dasar Binary Tree dan Graph sebelum melakukan implementasi program agar lebih mudah dalam memahami alur kerja struktur data tersebut.
2. Praktikan perlu lebih banyak berlatih membuat dan memodifikasi program Binary Tree maupun Graph untuk meningkatkan kemampuan pemrograman dan pemahaman algoritma.
3. Diperlukan ketelitian dalam menentukan hubungan antar node atau vertex agar struktur data yang dibangun sesuai dengan tujuan yang diinginkan.
4. Sebaiknya dilakukan pengembangan program yang lebih kompleks, seperti Binary Search Tree (BST), Depth First Search (DFS), dan Breadth First Search (BFS), untuk memperdalam pemahaman mengenai struktur data non-linear.
5. Praktikum berikutnya diharapkan dapat memberikan lebih banyak studi kasus penerapan Binary Tree dan Graph dalam kehidupan nyata sehingga mahasiswa dapat memahami manfaat dan penggunaannya secara lebih luas.

DAFTAR PUSTAKA

Telkom University Jakarta. 2024. *Pengenalan Tree dan Graph dalam Struktur Data*. Tersedia pada: <https://jakarta.telkomuniversity.ac.id/pengenalan-tree-dan-graph-dalam-struktur-data/>. Diakses tanggal 11 Juni 2026.

